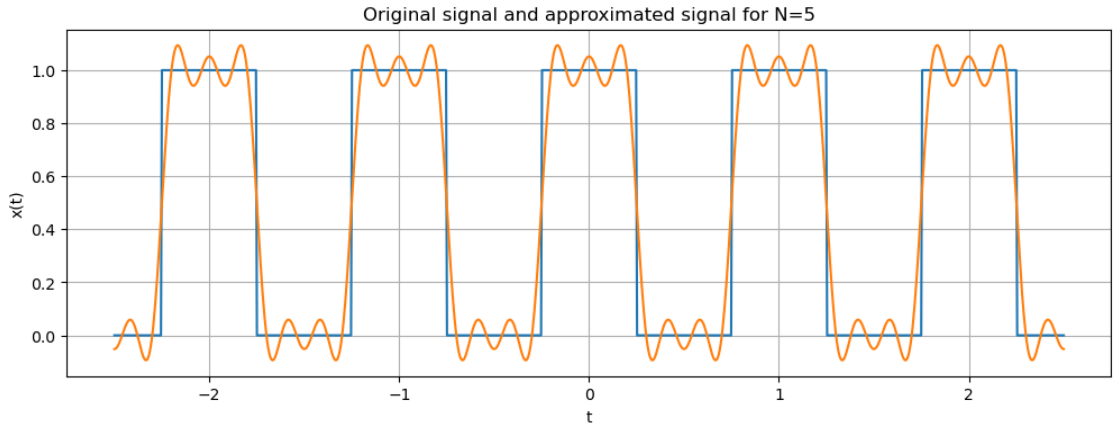
signal lab 2

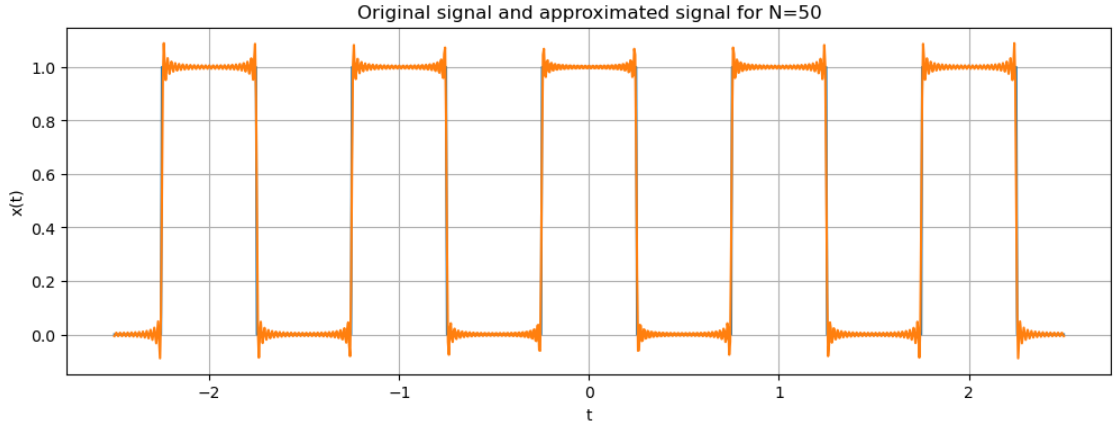
January 23, 2025

[3]:
```
# Imports
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft,fftshift,ifft
from scipy import signal

# Square pulse
def square(t):
    if t % 1 < 0.25 or t % 1 > 0.75:
        s=1
    elif t % 1 == 0.25 or t % 1 == 0.75:
        s = 0.5
    else:
        s=0
    return s
# Fourier series coefficients
def a(k):
 # Your code goes here
 # For k = 0 instance
    if k == 0:
 # a_k = Average value of the function
        a_k = 0.5
    else:
 # for k != 0 use coefficient formula
        a_k = 1 * np.sin(k * np.pi/2) / (k * np.pi)
    return a_k

def fs_approx(t, N):
    # Your code goes here
    x_t = 0.0

    # Parameters
    T = 1.0
    w0 = 2 * np.pi / T

    # Loop through -N to N and add individual complex exponentials
    for i in range(-N, N+1):
```

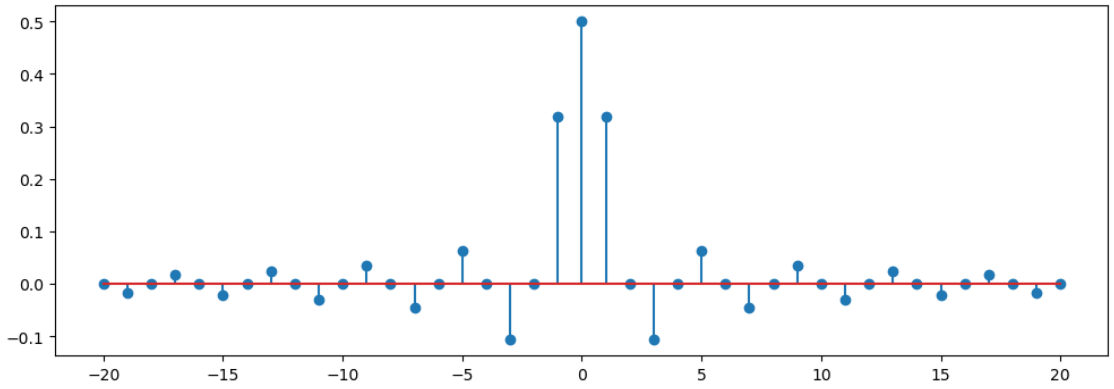
```
        x_t += (a(i)*np.e**(1j*i*w0*t)).real
    return x_t

# Fourier series approximation of the square wave
x = []
y = []
N = 5 # CHANGE HERE

# Creating a timestamp array
time = np.linspace(-2.5, 2.5,1000)

# Filling x and y arrays
for t in time:
 # Fill x array
 x.append(square(t))

 # Fill y array
 y.append(fs_approx(t,N))

# Plotting
fig,ax=plt.subplots(figsize=(12,4))

# Editing plot parameters
ax.plot(time,x)
ax.plot(time,y)
ax.grid()

ax.set_xlabel('t')
ax.set_ylabel('x(t)')
ax.title.set_text('Original signal and approximated signal for N=5')

# New values
x_2 = []
y_2 = []
N = 50

# Filling x and y arrays
for t in time:
 # Fill x array
 x_2.append(square(t))

 # Fill y array
 y_2.append(fs_approx(t,N))

# Plotting
fig,ax=plt.subplots(figsize=(12,4))
```

```
# Editing plot parameters
ax.plot(time,x_2)
ax.plot(time,y_2)
ax.grid()
ax.set_xlabel('t')
ax.set_ylabel('x(t)')
ax.title.set_text('Original signal and approximated signal for N=50')
```

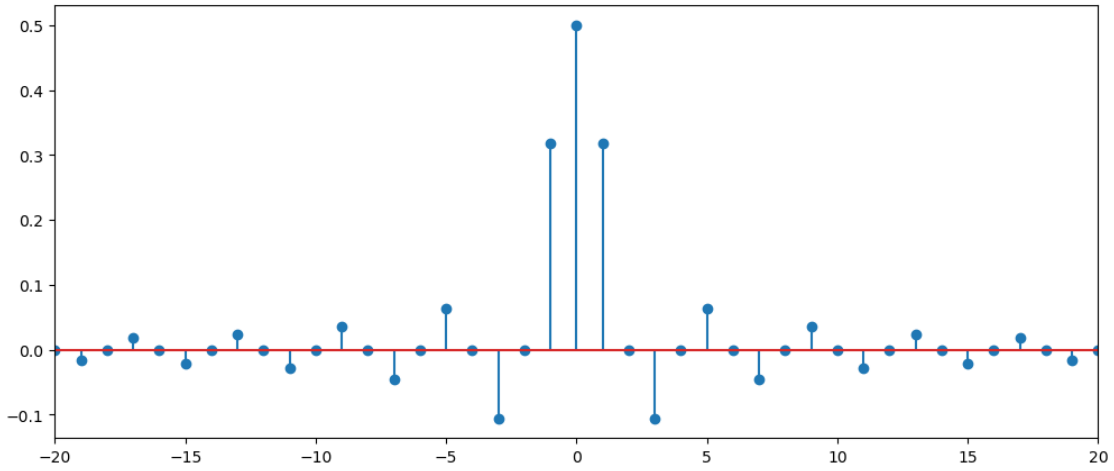




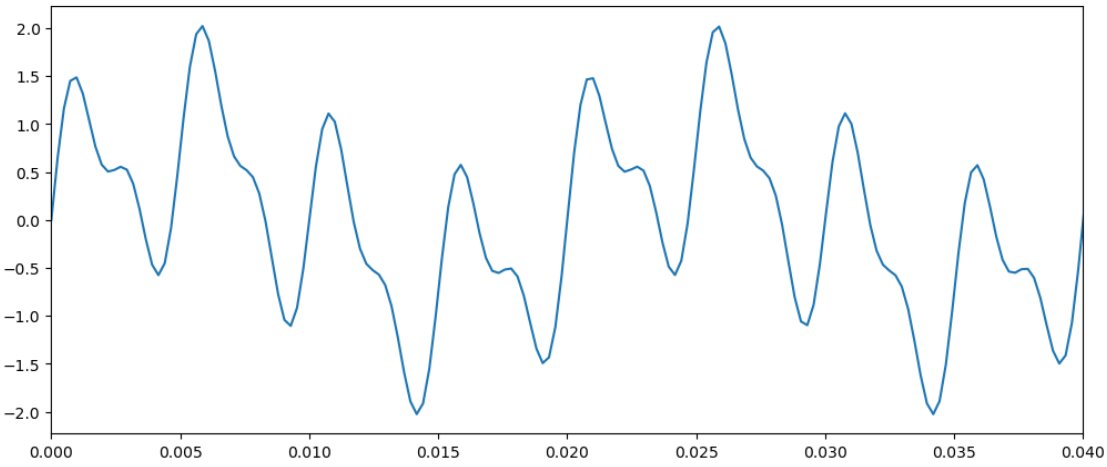
[5]:
```
# Creating two arrays
k = [i for i in range(-20,21)]
# Using list comprehension to get an array of a_k values for each k
ak = [a(ki) for ki in k]

# Creating stem plot
fig,ax=plt.subplots(figsize=(12,4))
ax.stem(k,ak);
```



[12]:
```
N = 200
t = np.linspace(0, 1-1/N, N)
x = []
for i in t:
    x.append(square(i))

# Obtaining FFT coefficients
X = fftshift(fft(x))
X_norm = X.real/N
k = np.linspace(-N/2, N/2-1, N)

# plotting fft coefficients
fig,ax=plt.subplots(figsize=(12,5))
# Plotting the FFT generated fourier coefficients against k values
ax.stem(k,X_norm);
ax.set_xlim(-20,20);
```

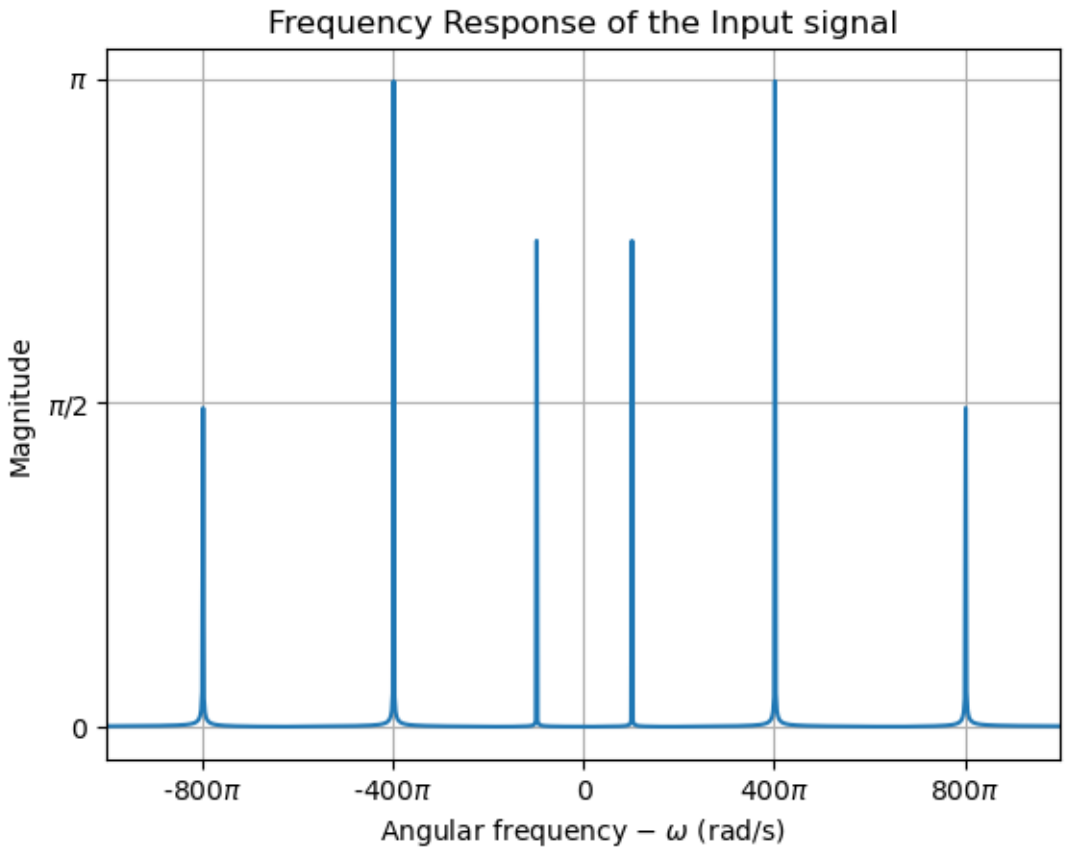


[14]:
```
# Creating 3 sinusoidal signals
# Your code goes here
w1 = 100 * np.pi
w2 = 400 * np.pi
w3 = 800 * np.pi

a1 = 0.75
a2 = 1
a3 = 0.5

fs = 4095
ws = 2*np.pi*fs


def x(t):
    # Your code goes here
    x_t = a1*np.sin(w1*t) + a2*np.sin(w2*t) + a3*np.sin(w3*t)
    return x_t

time = np.linspace(0,1,fs+1)
xt = [x(t_) for t_ in time]

# Plotting the input signal in time domain
# Your code goes here
fig,ax=plt.subplots(figsize=(12,5))
ax.plot(time,xt);
ax.set_xlim(0,0.04);
```



[17]:
```
# Fourier Transform of x(t)
Xw = fft(xt, 4096)*2*np.pi/fs
Xw = fftshift(Xw)
```

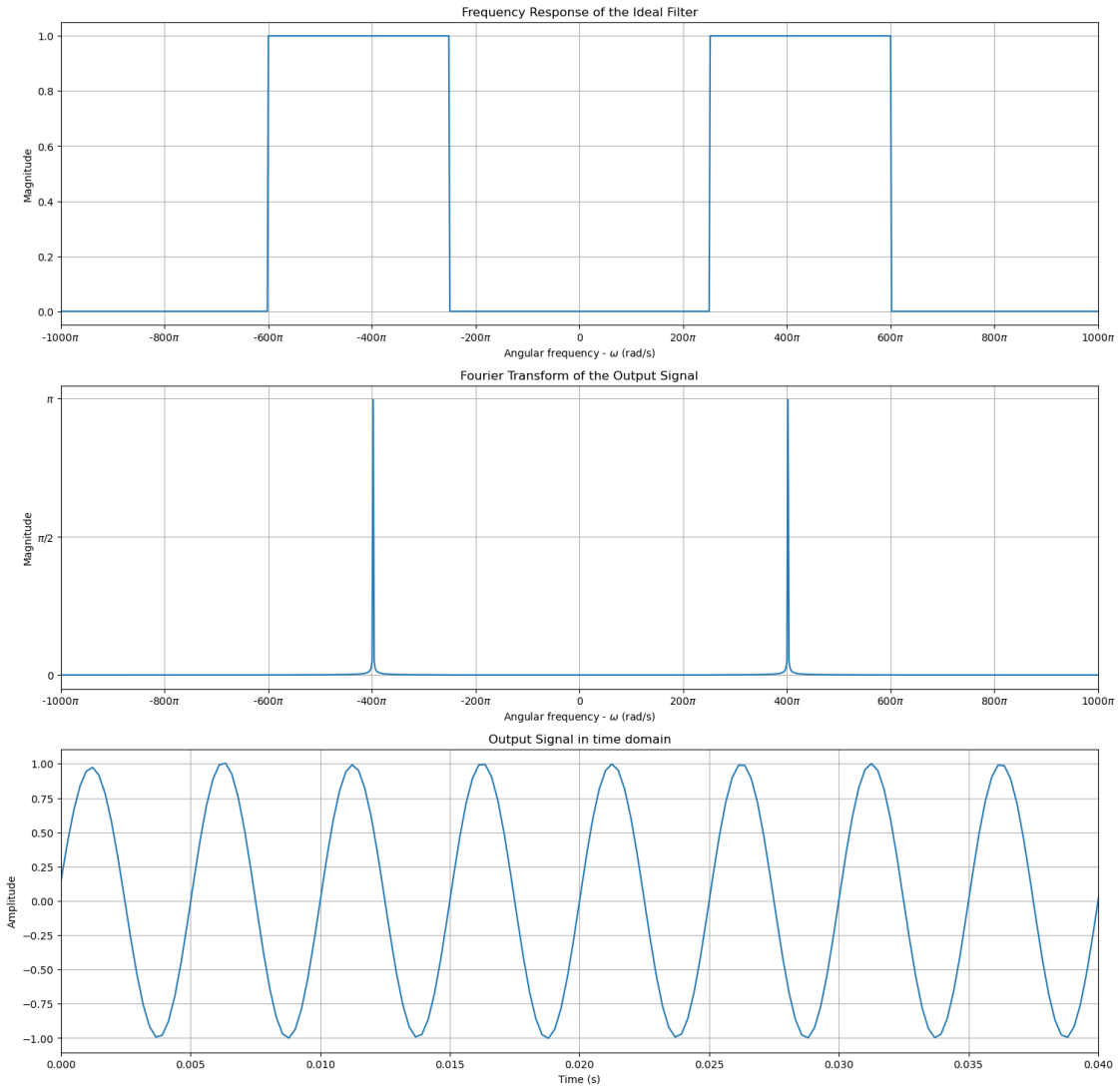
```
k = np.arange(1,4097)
w = k/4096*ws - ws/2

# Plotting the input signal in frequency domain
fig, ax = plt.subplots()
# Your code goes here
Xw_abs=np.abs(Xw)
ax.plot(w,Xw_abs)

# Editing plot attributes
ax.set_title('Frequency Response of the Input signal')
ax.set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
ax.set_ylabel('Magnitude')

ax.set_xticks(np.arange(-1200*np.pi, 1200*np.pi+1,400*np.pi))
ax.set_xticklabels([str(i)+(r'$\pi$' if i else '') for i in␣
 ↪range(-1200,1210,400)])
ax.set_xlim(-1000*np.pi, 1000*np.pi)
ax.set_yticks([0,np.pi/2,np.pi])
ax.set_yticklabels([0,r'$\pi$/2',r'$\pi$'])
plt.grid()
```



[20]:
```
# Ideal filter
wc1 = (w1+w2)/2
wc2 = (w2+w3)/2
def ideal_filter(w):
    # Your code goes here
    gain = 1
    if abs(w) <= wc1 or abs(w) >= wc2:
        gain = 0
    return gain
```

[21]:
```
k = np.arange(1,4097)
w = k/4096*ws - ws/2
# Your code goes here
H0w = [ideal_filter(i) for i in w]
# Simulation of Filtering
Y0w = np.multiply(Xw,H0w)
# Obtaining the time domain signal
y0t = ifft(fftshift(Y0w*fs/(2*np.pi)))
# Ideal filter frequency response (magnitude)
fig, axes = plt.subplots(3,1, figsize=(18,18))
axes[0].plot(w,H0w)
axes[0].set_title('Frequency Response of the Ideal Filter')
axes[0].set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
axes[0].set_ylabel('Magnitude')
axes[0].set_xticks(np.arange(-1200*np.pi, 1200*np.pi+1,200*np.pi))
axes[0].set_xticklabels([str(i)+(r'$\pi$' if i else '') for i in␣
 ↪range(-1200,1210,200)])
axes[0].set_xlim(-1000*np.pi, 1000*np.pi)
axes[0].grid()
# Frequency response of the ideal filter output (magnitude)
axes[1].plot(w,abs(Y0w))
axes[1].set_title('Fourier Transform of the Output Signal')
axes[1].set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
axes[1].set_ylabel('Magnitude')
axes[1].set_xticks(np.arange(-1200*np.pi, 1200*np.pi+1,200*np.pi))
axes[1].set_xticklabels([str(i)+(r'$\pi$' if i else '') for i in␣
 ↪range(-1200,1210,200)])
axes[1].set_xlim(-1000*np.pi, 1000*np.pi)
axes[1].set_yticks([0,np.pi/2,np.pi])
axes[1].set_yticklabels([0,r'$\pi$/2',r'$\pi$'])
axes[1].grid()
# Output signal in time domain
axes[2].plot(time,np.real(y0t))
axes[2].set_title('Output Signal in time domain')
```

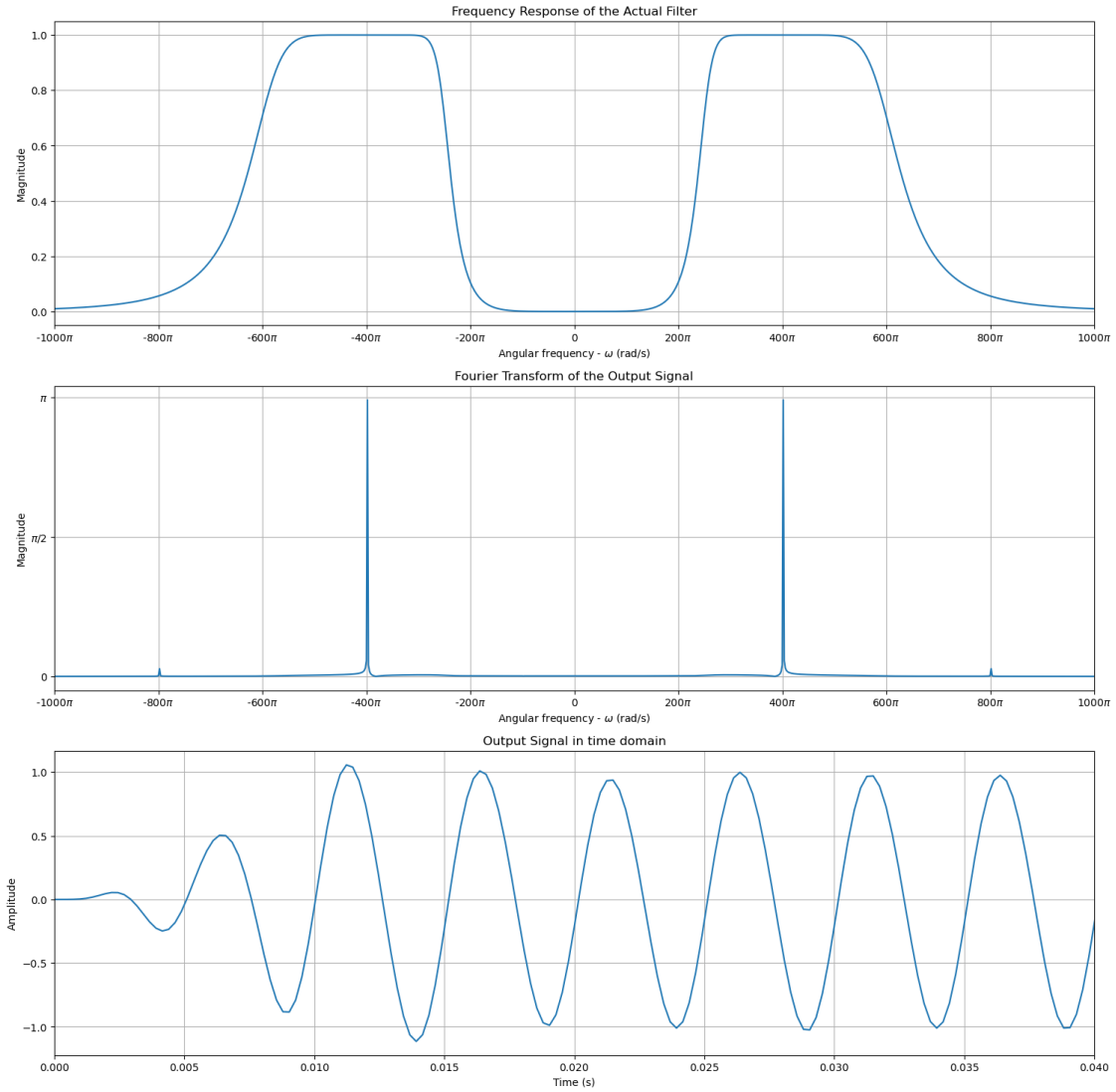
```
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Amplitude')
axes[2].set_xlim(0, 0.04)
axes[2].grid()
```



[19]:
```
# Actual Filter
b, a = signal.butter(5, [2*wc1/ws, 2*wc2/ws], 'bandpass', analog=False)
ww, h = signal.freqz(b, a, 2047)
ww = np.append(-np.flipud(ww), ww)*ws/(2*np.pi)
h = np.append(np.flipud(h), h)
# Filtering
y = signal.lfilter(b,a,xt)
# Obtaining the frequency response of the output signal
```

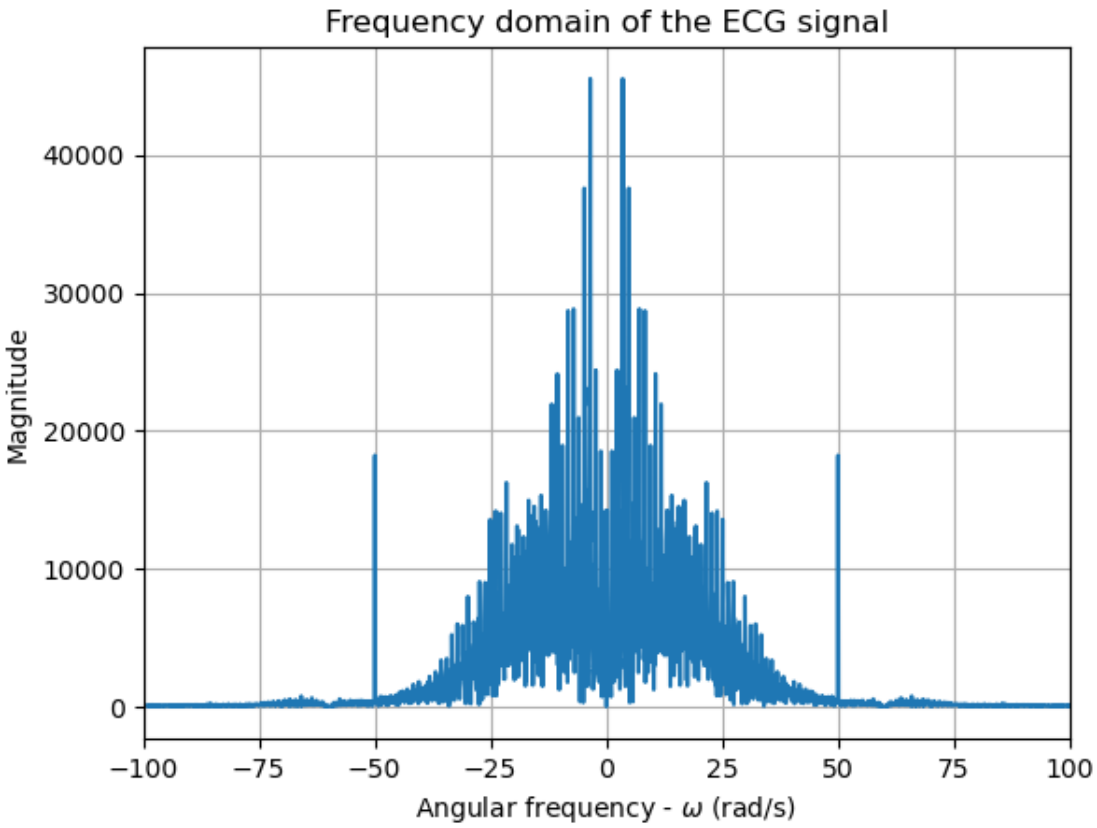
```
Y = fft(y,4096)*2*np.pi/fs
Y = fftshift(Y)
```

[21]:
```
# Actual filter frequency response (magnitude)
fig, axes = plt.subplots(3,1, figsize=(18,18))
axes[0].plot(ww, abs(h) )
axes[0].set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
axes[0].set_ylabel('Magnitude')
axes[0].set_title('Frequency Response of the Actual Filter')
axes[0].set_xticks(np.arange(-1200*np.pi, 1200*np.pi+1,200*np.pi))
axes[0].set_xticklabels([str(i)+(r'$\pi$' if i else '') for i in␣
 ↪range(-1200,1210,200)])
axes[0].set_xlim(-1000*np.pi, 1000*np.pi)
axes[0].grid()
# Frequency response of the actual filter output (magnitude)
axes[1].plot(w,abs(Y))
axes[1].set_title('Fourier Transform of the Output Signal')
axes[1].set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
axes[1].set_ylabel('Magnitude')
axes[1].set_xticks(np.arange(-1200*np.pi, 1200*np.pi+1,200*np.pi))
axes[1].set_xticklabels([str(i)+(r'$\pi$' if i else '') for i in␣
 ↪range(-1200,1210,200)])
axes[1].set_xlim(-1000*np.pi, 1000*np.pi)
axes[1].set_yticks([0,np.pi/2,np.pi])
axes[1].set_yticklabels([0,r'$\pi$/2',r'$\pi$'])
axes[1].grid()
# # Output signal in time domain
axes[2].plot(time,np.real(y))
axes[2].set_title('Output Signal in time domain')
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Amplitude')
axes[2].set_xlim(0, 0.04)
axes[2].grid()
```



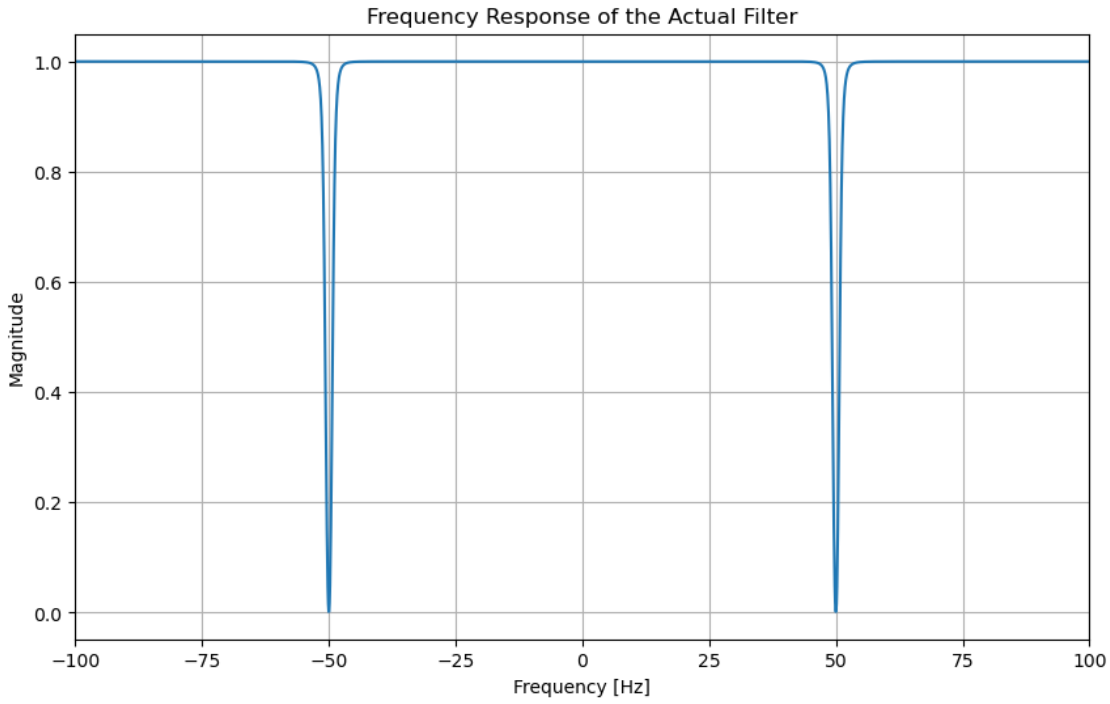
[24]:
```
import csv
# Reading the ECG data
ecg = []
# EDIT HERE
with open('ecg_signal.csv') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        ecg.append(float(row[0]))

duration = 10 # seconds
T =duration/len(ecg)
Fs = 1/T
# Obtaining the fourier transform
```

```
F =fftshift(fft(ecg))
fr = np.linspace(-Fs/2, Fs/2, len(F))

# Plotting the ecg signal in frequency domain
fig, ax = plt.subplots()
# Taking the absolute value and plotting
F_abs=np.abs(F)
ax.plot(fr,F_abs)
# Setting plot parameters
ax.set_title('Frequency domain of the ECG signal')
ax.set_xlabel('Angular frequency -'+r' $\omega$ (rad/s)')
ax.set_ylabel('Magnitude')
ax.set_xlim(-100, 100)
plt.grid()
```



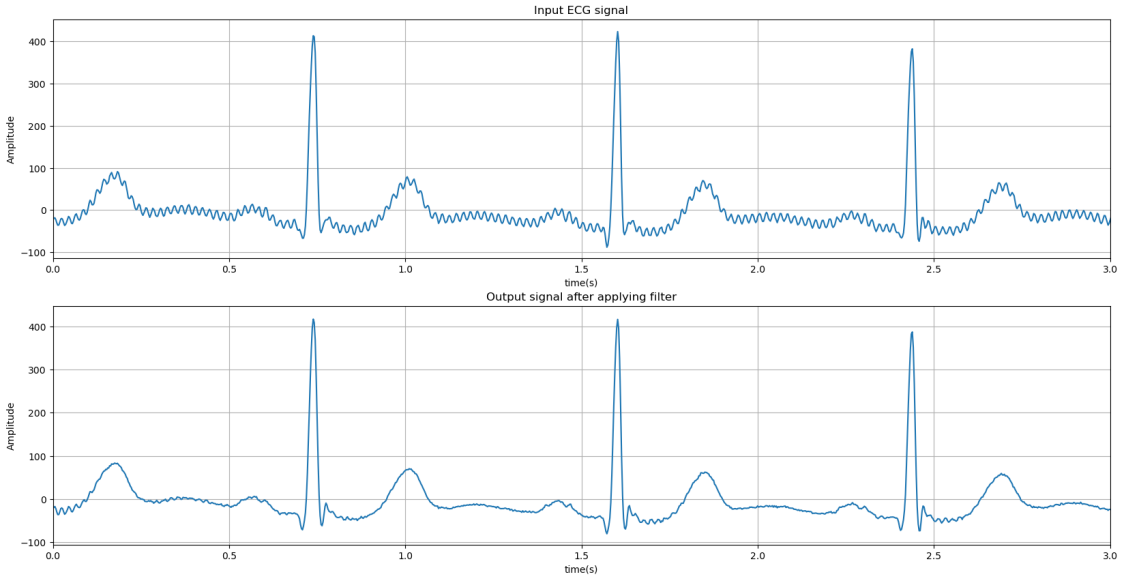
[29]:
```
# Designing the filter
f1 = 49
f2 = 51
filter_type = 'bandstop' # EDIT HERE
b, a = signal.butter(2, [2*f1/Fs, 2*f2/Fs], filter_type , analog=False)
```

```
# Obtaining the frequency response of the filter
ww,h=signal.freqz(b, a, 2047)
ww=np.append(-np.flipud(ww), ww)
h =np.append(np.flipud(h), h)
# Plotting the frequency response
fig, ax = plt.subplots(figsize=(10,6))
ax.plot(ww*Fs/(2*np.pi), abs(h) )
ax.set_title('Frequency Response of the Actual Filter')
ax.set_xlabel('Frequency [Hz]')
ax.set_ylabel('Magnitude')
ax.set_xlim(-100,100)
ax.grid()
```

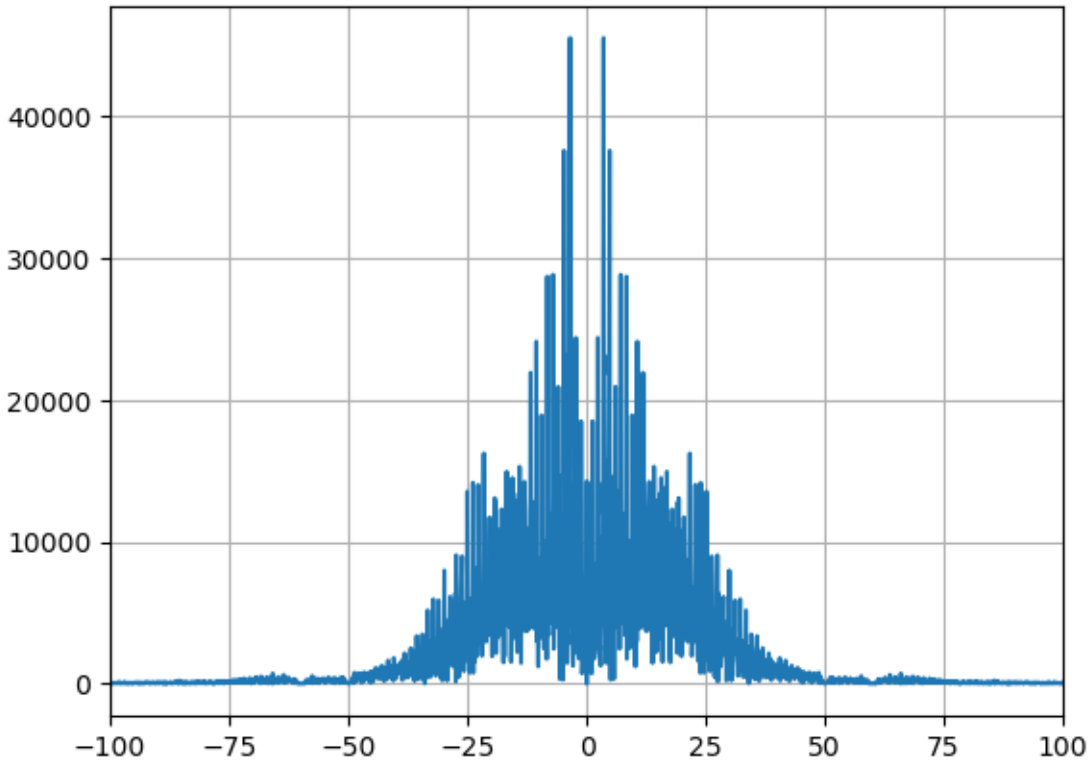


[32]:
```
time = np.arange(T, duration+T, T)
# Filtering the ECG waveform
output = signal.lfilter(b, a, ecg)
# Plotting
fig, axes = plt.subplots(2,1, figsize=(20,10));
axes[0].plot(time,ecg);
axes[1].plot(time,output);
# Limiting time scale
axes[0].set_xlim(0,3);
axes[1].set_xlim(0,3);
# Adding labels
```

```
axes[0].set_xlabel('time(s)')
axes[0].set_ylabel('Amplitude')
axes[1].set_xlabel('time(s)')
axes[1].set_ylabel('Amplitude')
# Adding titles
axes[0].set_title('Input ECG signal');
axes[1].set_title('Output signal after applying filter');
# Adding gridlines
axes[0].grid()
axes[1].grid()
```



[38]:
```
F =fftshift(fft(output))
fig, ax = plt.subplots()
ax.plot(fr,np.abs(F))
ax.set_xlim(-100,100)
ax.grid()
```


40000
30000
20000
10000
0
−100
−75
−50
−25
0
25
50
75
100

[ ]: